

```
>>> c.execute("SELECT * FROM workdata WHERE projname =
'projectA'")
<sqlite3.Cursor object at 0x0000000021C1B20>
>>> c.fetchone()
('01/01/2015', u'john', u'projectA', u'customerA', 8)
```

By default, we can access the data returned in a tuple by index based access. For example, in the code segment below, row variable with default factory binding, allows access to the different attributes by index offset. This requires us to hard code the index offset to map to a particular column name. And this would need to be changed if the position of the column changes.

```
>>> row = c.fetchone()
>>> print row[0]
01/01/2015
>>> print row[1]
john
>>> print row[2]
projectA
>>> print row[3]
customerA
```

By using *sqlite3.Row* binding we can access the same data in a more natural way using name binding. We can use the column name as the key to access the value fetched in the query, as shown in the following segment.

```
>>> import sqlite3
>>> conn = sqlite3.connect('test.db')
>>> conn.row_factory = sqlite3.Row
>>> c = conn.cursor()
...
>>> c.execute("SELECT * FROM workdata WHERE projname =
'projectA'")
<sqlite3.Cursor object at 0x00000000221EB20>
>>> row = c.fetchone()
>>> type(row)
<type 'sqlite3.Row'>
>>> print row["empname"]
john
```

We can see above that the *type* of record returned now is *sqlite3.Row* and this gives us access using the column name to retrieve the value. Before we do that though, let us insert the following values into the table using an *executemany()* function. This is an easy way to execute multiple SQLs in one statement.

```
>>> records = [('01/01/2015', 'miller', 'projectA', 'custome
rA', 8),
... ('01/01/2015', 'john', 'projectA', 'customerA', 8),
... ('02/01/2015', 'john', 'projectA', 'customerA', 8),
```

```
... ('02/01/2015', 'miller', 'projectB', 'customerB', 8)]
>>> c.executemany('INSERT into workdata
VALUES(?, ?, ?, ?)', records)
<sqlite3.Cursor object at 0x00000000224EB20>
>>> conn.commit()
```



Note: The recommended way for parameter substitution is using the “?” placeholder for parameters. Using Python string formatting methods leads to undesirable security vulnerabilities, like code injection through an SQL injection attack.

Now, using *sqlite3.Row* binding, one can write code to access data based on column name, for example, to sum up the hours spent on ‘projectA’, like in the code segment below.

```
>>> for row in c.execute("SELECT * FROM workdata WHERE
projname = 'projectA'"):
...     projectAhours+=row['hours']
...
>>> print "Time spent on ProjectA :"+ repr(projectAhours) +
" hours"
Time spent on ProjectA :24 hours
```

An alternate way of using the power of the SQL summation function is illustrated in the following code, which could be faster as the summary is being done in the database layer itself.

```
>>> c.execute("SELECT SUM(hours) as totalhours FROM workdata
where projname='pro
jectA'")
<sqlite3.Cursor object at 0x00000000221EB20>
>>> sumrow=c.fetchone()
>>> print "Time spent on ProjectA :"+
repr(sumrow['totalhours']) + " hours"
Time spent on ProjectA :24 hours
```



Note: The *repr* method used above to make *int* a printable string is similar in functionality to the *str* method, with one difference—*repr* gives an official representation while *str* gives an informal representation. For example, *repr* prints strings with quotes in the output, while *str* does not.

Working with a MySQL database to connect and run the same program

Having worked with SQLite3, now is a good time to move to MySQL, which vies for the title ‘The world’s most popular open source database’. In the current section, I assume that you have installed the MySQL database. For the purpose of this column, I have installed MySQL community edition (5.6.24 MySQL Community Server (GPL)). Also, for the sake of simplicity, I will be showing MySQL